

Colab, or "Colaboratory", allows you to write and execute Python in your browser, with

- Zero configuration required
- Access to GPUs free of charge
- Easy sharing

Whether you're a **student**, a **data scientist** or an **AI researcher**, Colab can make your work easier. Watch [Introduction to Colab](#) or [Colab Features You May Have Missed](#) to learn more, or just get started below!

✓ Getting started

The document you are reading is not a static web page, but an interactive environment called a **Colab notebook** that lets you write and execute code.

For example, here is a **code cell** with a short Python script that computes a value, stores it in a variable, and prints the result:

```
seconds_in_a_day = 24 * 60 * 60
seconds_in_a_day
```

```
86400
```

To execute the code in the above cell, select it with a click and then either press the play button to the left of the code, or use the keyboard shortcut "Command/Ctrl+Enter". To edit the code, just click the cell and start editing.

Variables that you define in one cell can later be used in other cells:

```
seconds_in_a_week = 7 * seconds_in_a_day
seconds_in_a_week
```

```
604800
```

Colab notebooks allow you to combine **executable code** and **rich text** in a single document, along with **images**, **HTML**, **LaTeX** and more. When you create your own Colab notebooks, they are stored in your Google Drive account. You can easily share your Colab notebooks with co-workers or friends, allowing them to comment on your notebooks or even edit them. To learn more, see [Overview of Colab](#). To create a new Colab notebook you can use the File menu above, or use the following link: [create a new Colab notebook](#).

Colab notebooks are Jupyter notebooks that are hosted by Colab. To learn more about the Jupyter project, see [jupyter.org](#).

✓ Data science

With Colab you can harness the full power of popular Python libraries to analyze and visualize data. The code cell below uses **numpy** to generate some random data, and uses **matplotlib** to visualize it. To edit the code, just click the cell and start editing.

You can import your own data into Colab notebooks from your Google Drive account, including from spreadsheets, as well as from Github and many other sources. To learn more about importing data, and how Colab can be used for data science, see the links below under [Working with Data](#).

```
import numpy as np
import cv2
import matplotlib.pyplot as plt

# 1. Synthesize a Fake Surveillance Stream Frame
# We generate a baseline background scene (a security corridor) and an intruding object.
grid_size = 512
background_scene = np.ones((grid_size, grid_size), dtype=np.uint8) * 100
# Add structural background noise/lines to represent a physical hallway
cv2.line(background_scene, (0, 100), (512, 400), 50, 3)
cv2.line(background_scene, (512, 100), (0, 400), 50, 3)

# Add baseline high-frequency camera sensor noise to the raw environment
raw_sensor_noise = np.random.normal(0, 15, background_scene.shape).astype(np.int16)
live_frame = np.clip(background_scene.astype(np.int16) + raw_sensor_noise, 0, 255).astype(np.uint8)

# Insert a dynamic moving object (simulating an unauthorized intruder entering the zone)
# The intruder is placed inside a designated "Restricted Red Zone"
intruder_center = (260, 280)
cv2.circle(live_frame, intruder_center, 25, 220, -1) # High contrast intruder

# 2. LOW-LEVEL VISION: Sensor Denoising and Structural Filtering
# Goal: Condition raw pixel values to clean up sensor noise without destroying edges.
low_level_clean = cv2.GaussianBlur(live_frame, (7, 7), 0)
```

```

# 3. MID-LEVEL VISION: Object Localization and Bounding Feature Extraction
# Goal: Isolate the moving entity from static background data and map its geometry.
# Compute raw pixel difference to isolate the change
frame_delta = cv2.absdiff(background_scene, low_level_clean)
_, threshold_map = cv2.threshold(frame_delta, 35, 255, cv2.THRESH_BINARY)

# Extract contours (geometric features) out of the binary pixel mask
contours, _ = cv2.findContours(threshold_map, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

# Prepare an RGB display frame to visualize tracking features
mid_level_output = cv2.cvtColor(low_level_clean, cv2.COLOR_GRAY2RGB)
detected_box = None

for contour in contours:
    if cv2.contourArea(contour) > 200: # Filter out residual noise spikes
        (x, y, w, h) = cv2.boundingRect(contour)
        detected_box = (x, y, w, h)
        # Draw bounding box feature mapping around the object
        cv2.rectangle(mid_level_output, (x, y), (x + w, y + h), (0, 255, 0), 2)

# 4. HIGH-LEVEL VISION: Semantic Reasoning and Threat Level Evaluation
# Goal: Cross-reference extracted tracking features with logical context to trigger an alert.
high_level_output = cv2.cvtColor(low_level_clean, cv2.COLOR_GRAY2RGB)

# Define a semantic restriction zone polygon layout
restricted_zone_poly = np.array([[150, 200], [362, 200], [450, 450], [62, 450]], np.int32)
cv2.polylines(high_level_output, [restricted_zone_poly], True, (255, 255, 0), 2)
cv2.putText(high_level_output, "CRITICAL RESTRICTED ZONE", (70, 190), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 0), 2)

system_alert = "STATUS: SECURE (NO THREATS)"
alert_color = (0, 255, 0) # Safe Green

if detected_box is not None:
    bx, by, bw, bh = detected_box
    object_centroid = (bx + bw//2, by + bh//2)

    # Mathematical Semantic Verification: Is the isolated feature inside the restricted region boundary?
    is_inside = cv2.pointPolygonTest(restricted_zone_poly, object_centroid, False)

    if is_inside >= 0:
        system_alert = "CRITICAL ALERT: UNAUTHORIZED INTRUSION DETECTED!"
        alert_color = (255, 0, 0) # Threat Red
        # Highlight threat target red
        cv2.rectangle(high_level_output, (bx, by), (bx + bw, by + bh), (255, 0, 0), 3)
        cv2.circle(high_level_output, object_centroid, 5, (255, 0, 0), -1)

# Overlay semantic system decision directly onto the display frame
cv2.putText(high_level_output, system_alert, (15, 40), cv2.FONT_HERSHEY_SIMPLEX, 0.6, alert_color, 2)

# 5. Plot and Compare the Surveillance Architecture Stages
plt.figure(figsize=(20, 10))

# Plot Raw Video Capture (Low Level Input)
plt.subplot(1, 4, 1)
plt.imshow(live_frame, cmap='gray')
plt.title("1. Raw Surveillance Stream\n(Contains Sensor Noise)", fontsize=11)
plt.axis('off')

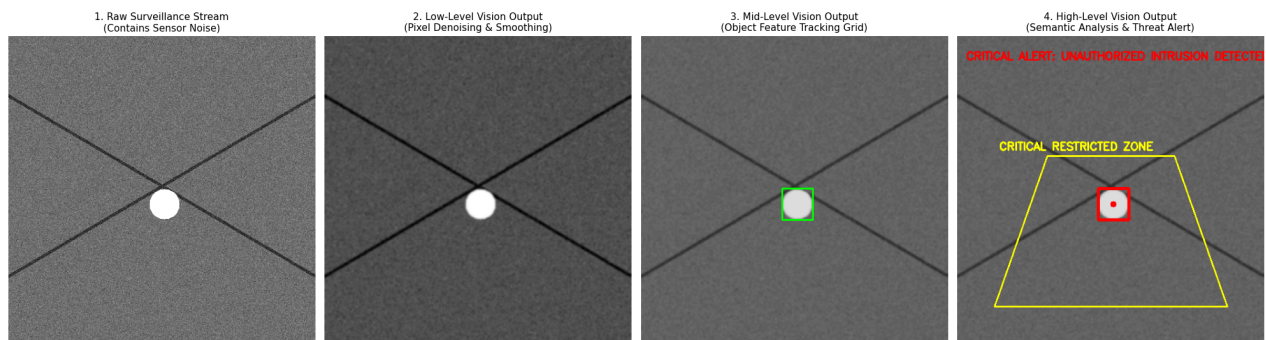
# Plot Low-Level Stream Conditioning Output
plt.subplot(1, 4, 2)
plt.imshow(low_level_clean, cmap='gray')
plt.title("2. Low-Level Vision Output\n(Pixel Denoising & Smoothing)", fontsize=11)
plt.axis('off')

# Plot Mid-Level Bounding Box Detections
plt.subplot(1, 4, 3)
plt.imshow(mid_level_output)
plt.title("3. Mid-Level Vision Output\n(Object Feature Tracking Grid)", fontsize=11)
plt.axis('off')

# Plot High-Level Semantic Security Context Decision
plt.subplot(1, 4, 4)
plt.imshow(high_level_output)
plt.title("4. High-Level Vision Output\n(Semantic Analysis & Threat Alert)", fontsize=11)
plt.axis('off')

plt.tight_layout()
plt.show()

```



Colab notebooks execute code on Google's cloud servers, meaning you can leverage the power of Google hardware, including [GPUs](#) and [TPUs](#), regardless of the power of your machine. All you need is a browser.

For example, if you find yourself waiting for **pandas** code to finish running and want to go faster, you can switch to a GPU Runtime and use libraries like [RAPIDS cuDF](#) that provide zero-code-change acceleration.

To learn more about accelerating pandas on Colab, see the [10 minute guide](#) or [US stock market data analysis demo](#).

Machine learning

With Colab you can import an image dataset, train an image classifier on it, and evaluate the model, all in just [a few lines of code](#).

Colab is used extensively in the machine learning community with applications including:

- Getting started with TensorFlow
- Developing and training neural networks
- Experimenting with TPUs
- Disseminating AI research
- Creating tutorials

To see sample Colab notebooks that demonstrate machine learning applications, see the [machine learning examples](#) below.

More Resources

Working with Notebooks in Colab

- [Overview of Colab](#)
- [Guide to Markdown](#)
- [Importing libraries and installing dependencies](#)
- [Saving and loading notebooks in GitHub](#)
- [Interactive forms](#)
- [Interactive widgets](#)

Working with Data

- [Loading data: Drive, Sheets, and Google Cloud Storage](#)
- [Charts: visualizing data](#)
- [Getting started with BigQuery](#)

Machine Learning

These are a few of the notebooks related to Machine Learning, including Google's online Machine Learning course. See the [full course website](#) for more.

- [Intro to Pandas DataFrame](#)
- [Intro to RAPIDS cuDF to accelerate pandas](#)
- [Getting Started with cuML's accelerator mode](#)

Using Accelerated Hardware